Design a Webhook Platform

Webhook 是一種向外發送的 HTTP callback：當某個事件發生時，服務會主動對**你的** URL 發送請求——也就是「**event → POST 到你的 endpoint**」。

它讓系統之間能以 **push** 的方式傳遞更新，而不需要不斷 polling。

例如：Stripe 提供 webhooks，讓使用者可以訂閱 `invoice.created`、`invoice.paid` 等 events；GitHub 也提供 webhooks，讓使用者接收 `deployment`、`pull_request` 等 events。

你正在為一個大型 SaaS 建立一個 **multi-tenant webhook platform（多租戶 Webhook 平台）**。

內部的 product services 會產生 domain events（例如 `invoice.paid`、`issue.opened`）。

客戶可以註冊 **endpoints（接收端點）**，選擇想接收的 events，平台需以 HTTP request 的形式將事件傳送過去。

此平台必須能**接收（ingest）內部事件，並可靠地 fan-out** 到客戶註冊的 endpoints。

功能性需求（Functional Requirements）

- 使用者應該能夠管理其 endpoints（啟用 activate、停用 deactivate），以及希望接收哪些 event types。
- 內部 services 會將 events 發送至 webhook ingestion platform。
- Webhook ingestion platform 需將 events 轉換為 webhooks，並投遞至客戶已註冊的 endpoints。

非功能性需求（Non-Functional Requirements）

- 規模需求：1 億（100M）使用者、100 種 event types。
- 近即時（near real-time）投遞，並提供 at-least-once delivery guarantee。
- 若客戶 endpoints 暫時無法使用，系統仍需嘗試在數天內完成投遞。

buildmoat.org

API 設計（API Design）

1. 註冊端點（Register Endpoints）

```
POST
v1/webhook
{
  event_type,
  endpoint
}
{
  id
}
```

- `event_type`（事件類型）：使用者希望訂閱的 domain event。
- `endpoint`（接收端點）：用來接收 webhook 的 HTTP URL。
- `id`（端點 ID）：系統建立並回傳的唯一識別碼，用於後續管理。

2. 管理端點（Manage Endpoints）

更新端點（Update Endpoint）

```
PUT
v1/webhook/:id
{
  endpoint
}
```

- `id`：要修改的 webhook endpoint 識別碼。
- `endpoint`：更新後的接收 URL 或設定內容。

3. 刪除端點（Delete Endpoint）

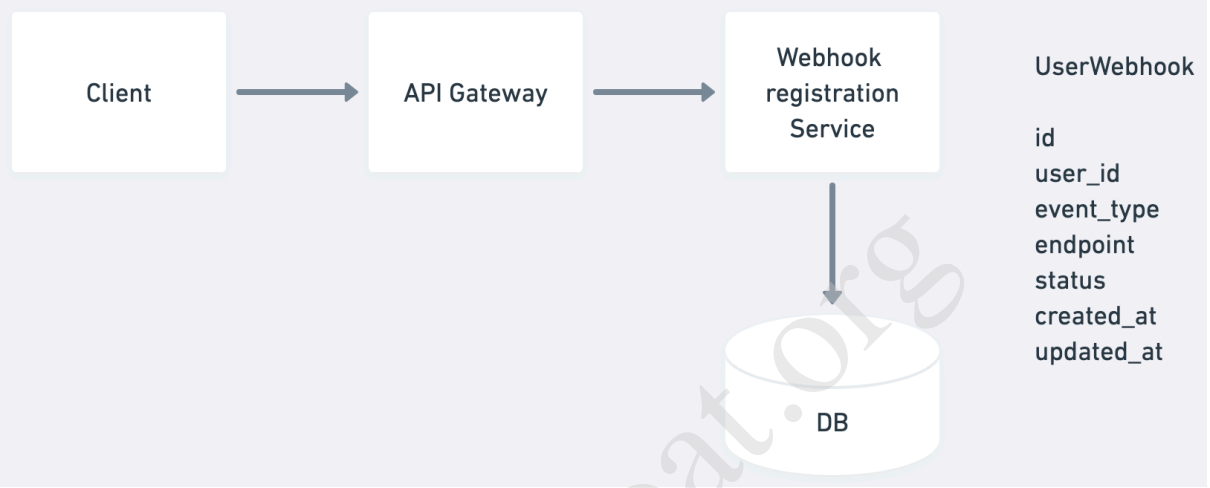
```
DELETE
v1/webhook/:id
```

buildmoat.org

- `id`：要刪除的 webhook endpoint 識別碼。

High-Level Design

1. 使用者端點管理流程（User Endpoints Management Flow）



Webhook 端點註冊（Webhook endpoint registration）

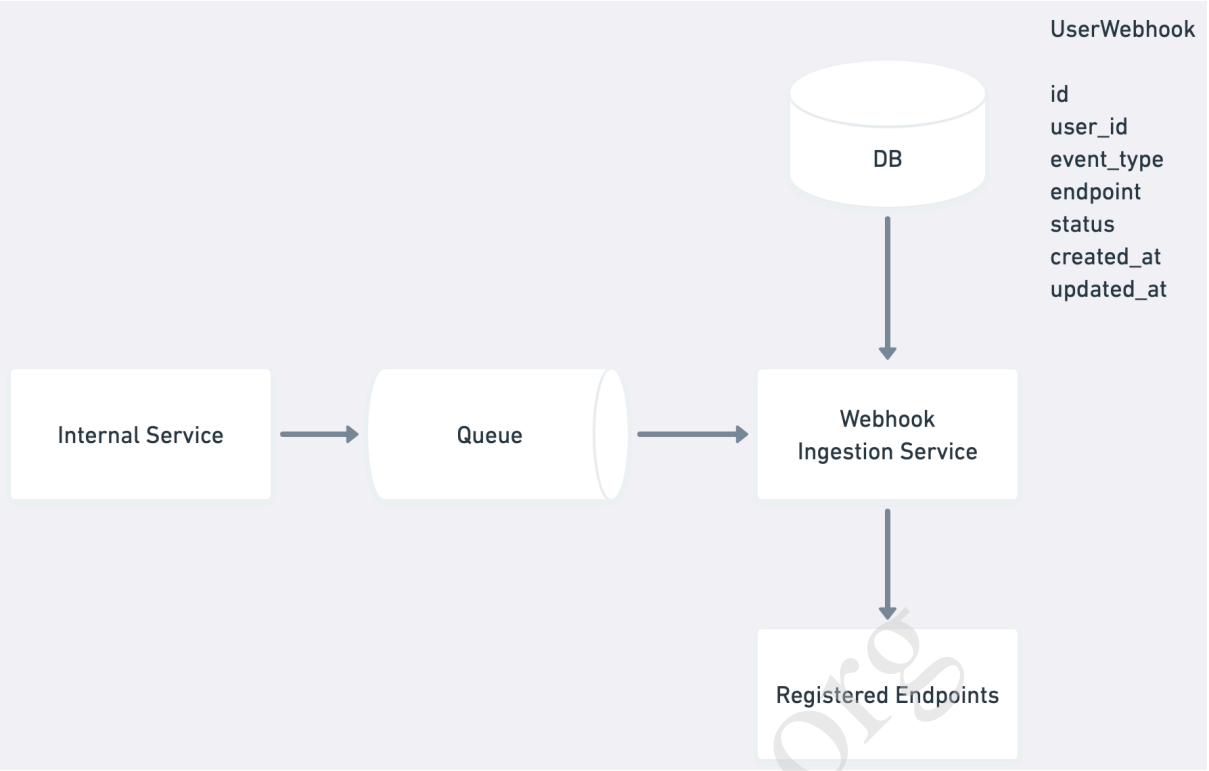
- 使用者呼叫 `POST v1/webhook`，並提供 `event_type` 與 `endpoint`，以建立對應 event_type 的 endpoint。
- Server 在資料庫（DB）中建立一筆紀錄，將 endpoint 設定持久化。
- Server 回傳 endpoint ID 以及相關的 metadata。

Webhook 端點編輯 / 刪除（Webhook endpoint edit / deletion）

- 使用者呼叫 `PUT v1/webhook/:id` 或 `DELETE v1/webhook/:id`，以更新或刪除 endpoint。

2. Webhook 投遞流程（Webhook Delivery Flow）

buildnoat.org



- 內部服務（internal service）將 internal events 發佈到 Message Queue（訊息佇列）。
- Webhook ingestion service 從 queue 中拉取（poll）events，並查詢 DB，確認是否存在對應的 endpoint（依 `(user_id, event_type)`）。
- 在解析出 endpoint 後，ingestion service 會以 HTTP POST 的方式，將 event 發送至該 endpoint。
- 我們在 internal services 與 ingestion service 之間加入一層 queue：
  - **解耦（Decoupling）**：ingestion service 的失敗不會影響 internal service 的正常流程。
  - **緩衝（Buffer）**：即使 event 流量突增，也不會直接壓垮 ingestion service。

深入探討（Deep Dive）

1. 我們如何實現至少一次投遞（at-least-once delivery）？

如果在投遞事件時，使用者的 endpoint 回傳錯誤或無法連線，該怎麼辦？

buildmoat.org

我們可以先採用 **exponential backoff with jitter（指數退避加隨機抖動）**，在達到初始重試上限前進行多次 retry。

這是分散式系統中非常典型的 retry strategy，原因包括：

- **在系統異常時快速退讓（Back off fast when things are broken）**

  當某個服務或網路連線出現問題時，立刻重試（或使用固定間隔）只會增加系統負載，甚至引發連鎖故障。

  指數退避（例如 100ms、200ms、400ms、800ms...）可以快速降低 retry 壓力，讓系統有時間恢復。

- **避免同步重試風暴（Avoid synchronized retry storms）**

  若成千上萬的 client 同時失敗，並按照相同節奏重試，會形成「thundering herd」。

  Jitter（隨機化）能將 retries 分散在不同時間點，使 server 承受較平滑、可控的負載。

如果在耗盡所有 retries 後，endpoint 仍然回傳錯誤呢？

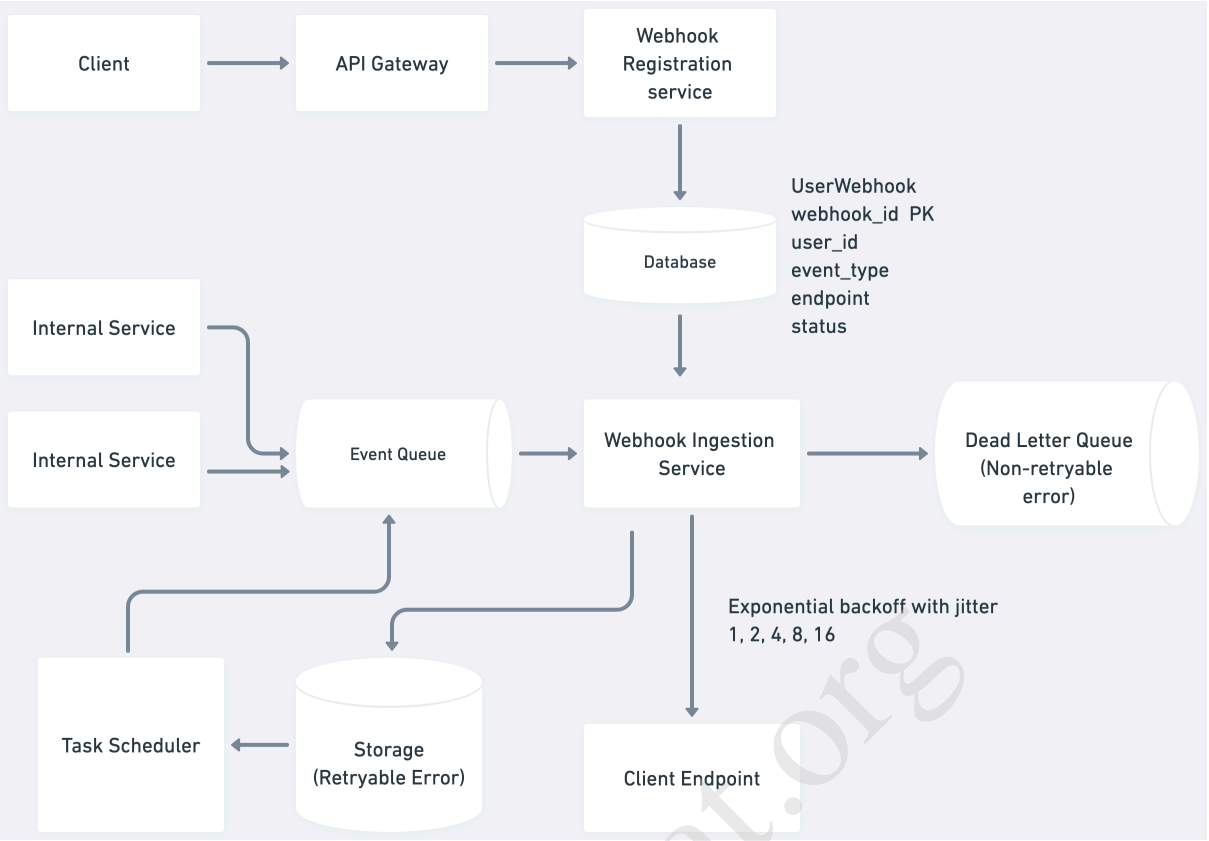
在初始 retries（例如 5 次）失敗後，我們可以將這些 events 標記為 retryable error，並寫入 persistent storage。

接著由一個 scheduler 週期性地掃描到期事件，並在稍後重新放回 queue。

若在服務 SLA 範圍內（例如承諾最長 retry 3 天）仍然失敗，我們可以將事件送入 DLQ（Dead Letter Queue），同時將 endpoint 狀態標記為 suspended，並通知使用者。

若某些事件一開始就被判定為 non-retryable（例如 schema 錯誤、缺少 secret 等），則會直接送入 DLQ，不進行 retries。

buildmoat.org



2. 系統擴展性（Scaling the system）

Endpoint DB

假設每個 `(user, event_type)` 每天會送出 1,000 筆 events：

```
1000 (events) *100M (users) *100 (event_type) =10Tevents / day
```

```
10T /86400 ≈120Mevents /sec
```

- DB 的存取模式為 **讀取遠大於寫入（read >> writes）**。

  使用者只在註冊 endpoint 時寫入一次，但 ingestion service 在每次事件到達時都需要查詢。

  為了保護 DB 並降低 ingestion latency，我們可以在 DB 前加一層 cache。

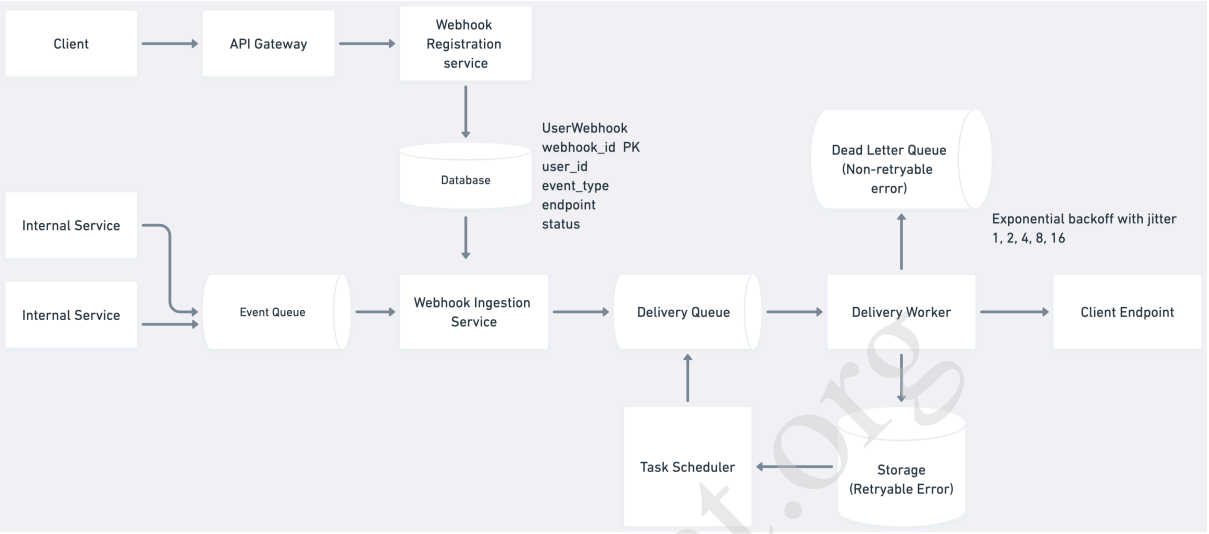
  - Key：`user_id:event_type`
  - Value：對應的 endpoint 設定
- 假設 DB 中每一列為 1KB，

buildmoat.org

`100M users * 100 event_type = 10B rows ≈ 10TB`，在現代 SSD 上是可行的。

若遇到硬體瓶頸，可再進行 sharding；但在設計層面上，單一 DB instance 已可作為起點。

Queues



- 為了解耦 webhook ingestion 與 event delivery，我們引入了專用的 delivery workers，並在 ingestion service 與 workers 之間加入 queue。
  - Queue 作為「送出嘗試（send attempts）」的 durable buffer（包含 retries），即使 worker crash 或 autoscale，任務也不會遺失。
  - 慢速或失敗的 receivers 不會阻塞 ingestion。
  - Delivery workers 可獨立於 ingestion service 進行水平擴展。
- 為了承受高事件吞吐量（120M events/sec），我們需要對 queue 進行 partition：
  - Event queue 的 partition key 可使用 `event_id` 以平均分散負載。
  - Delivery queue 的 partition key 可使用 `endpoint_id`，確保同一 endpoint 的 deliveries 進入同一 partition，方便 per-endpoint throttling 與 circuit breaking。
  - 若某 endpoint 成為 hot spot，可進一步做 sub-sharding，例如 `endpoint_id + ":" + hash(event_id) % K`，其中 K 為 shard 數量。
- 為了實現 at-least-once delivery，可使用 Amazon SQS，透過 visibility timeout 機制，確保 event 在被 consumer 成功處理後才會從 queue 中移除。

buildmoat.org

3. 驗證與安全（Authentication）

我們要如何確認使用者真的擁有該 endpoint，而不是隨意填一個 URL 來濫用系統？

1. **TLS server authentication（基礎且必須）**

   僅允許傳送至 `https://…`，並驗證 server certificate 與 hostname。

   - 驗證憑證鏈是否由可信任的 CA 簽發。
   - 驗證憑證中的 hostname（CN / SAN）是否與 `receiver.com` 相符。
   - 若皆通過，代表我們確實是在與該 domain 的擁有者通訊。

2. **Endpoint ownership proof at registration（常見作法）**

   - 執行 **challenge / response**：

     我們先向 receiver POST 一個隨機 token，只有在對方正確回傳該 token 後，才接受該 endpoint。

   - 可在關鍵變更（domain、IP、certificate）或定期重新進行 challenge。

我們要如何向 webhook consumer 證明「我們確實是合法的 sender」？

通常透過 **payload signing（簽章）** 來實現：

- 在 subscriber 註冊 endpoint 時，配置一組 shared secret。
- Sender 與 receiver 在註冊階段即共享該 secret。
- 每次 webhook 投遞時，sender 使用該 secret，對 payload（通常是 `timestamp + raw_body`）進行 HMAC-SHA256 簽章。
- Receiver 以相同方式重新計算 HMAC，並進行 **constant-time comparison** 以防止 timing attack。

buildmoat.org